

```

import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import numpy as np
import pandas as pd
import seaborn as sns

# Definição dos vetores
v1 = np.array([3, 2, 5])
v2 = np.array([1, 4, 2])
v3 = np.array([-2, 6, 3])

# Operações de vetores
soma_vetores = v1 + v2
subtracao_vetores = v2 - v1

escalar = 2
multiplicacao_escalar = v1 * escalar

produto_escalar = np.dot(v1, v2)

# Cálculo do ângulo entre v1 e v2
cos_theta = np.dot(v1, v2) / (np.linalg.norm(v1) * np.linalg.norm(v2))
ang_rad = np.arccos(np.clip(cos_theta, -1.0, 1.0)) # Limitar para evitar erros de ponto flutuante
ang_graus = np.degrees(ang_rad)

produto_vetorial = np.cross(v1, v2)

vetor_unitario_v1 = v1 / np.linalg.norm(v1)

# Helper function para plotagem de vetores 3D
def plot_vectors_3d(vectors_data, title="Visualização de Vetores 3D",
                    x_label="X", y_label="Y", z_label="Z"):
    fig = plt.figure(figsize=(10, 8))
    ax = fig.add_subplot(111, projection='3d')

    max_coord = 0
    for vec, color, label in vectors_data:
        ax.quiver(0, 0, 0, vec[0], vec[1], vec[2], color=color,
                  label=label, arrow_length_ratio=0.08, linewidth=2)
        max_coord = max(max_coord, np.max(np.abs(vec))) if len(vec) > 0 else max_coord

    # Ajusta os limites para que todos os vetores sejam visíveis
    limit = max_coord * 1.2 if max_coord > 0 else 1

```

```

ax.set_xlim([-limit, limit])
ax.set_ylim([-limit, limit])
ax.set_zlim([-limit, limit])
ax.set_xlabel(x_label)
ax.set_ylabel(y_label)
ax.set_zlabel(z_label)
ax.set_title(title)
ax.legend()
ax.view_init(elev=20, azim=30) # Um bom ângulo de visão padrão
plt.grid(True)
plt.show()

```

Visualização das Operações

Criando dados para o gráfico de barras (magnitudes)

```

magnitudes = [
    {"vetor": "v1", "magnitude": np.linalg.norm(v1)},
    {"vetor": "v2", "magnitude": np.linalg.norm(v2)},
    {"vetor": "v3", "magnitude": np.linalg.norm(v3)}
]

```

```

df_magnitudes = pd.DataFrame(magnitudes)

```

```

plt.figure(figsize=(8, 5))
sns.barplot(x='vetor', y='magnitude', data=df_magnitudes, palette='viridis', hue='vetor',
            legend=False)
plt.title('Magnitude dos Vetores v1, v2 e v3')
plt.xlabel('Vetor')
plt.ylabel('Magnitude')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()

```

1. Adição de Vetores ($v_1 + v_2$)

Visualização da Adição de Vetores

```

vectors_to_plot = [
    (v1, 'r', 'Vetor v1'),
    (v2, 'g', 'Vetor v2'),
    (soma_vetores, 'b', 'Soma (v1 + v2)')
]
plot_vectors_3d(vectors_to_plot, title='Adição de Vetores: v1 + v2')

```

```
# ##### 2. Subtração de Vetores ( $v_2 - v_1$ )
```

```
# Visualização da Subtração de Vetores
```

```
vectors_to_plot = [  
    (v1, 'r', 'Vetor v1'),  
    (v2, 'g', 'Vetor v2'),  
    (subtracao_vetores, 'b', 'Subtração ( $v_2 - v_1$ )')  
]  
plot_vectors_3d(vectors_to_plot, title='Subtração de Vetores:  $v_2 - v_1$ )
```

```
# ##### 3. Multiplicação por um Escalar ( $v_1 * 2$ )
```

```
# Visualização da Multiplicação por um Escalar
```

```
vectors_to_plot = [  
    (v1, 'r', 'Vetor v1'),  
    (multiplicacao_escalar, 'g', f'V1 * {escalar}')  
]  
plot_vectors_3d(vectors_to_plot, title=f'Multiplicação por Escalar:  $v_1 * \{escalar\}$ )
```

```
# ##### 4. Produto Escalar ( $v_1 \cdot v_2$ )
```

```
# Visualização do Produto Escalar (mostrar vetores e ângulo)
```

```
vectors_to_plot_2d = [  
    (v1, 'r', 'Vetor v1'),  
    (v2, 'g', 'Vetor v2')  
]  
plot_vectors_3d(vectors_to_plot_2d, title=f'Produto Escalar:  $v_1 \cdot v_2 = \{produto\_escalar:.2f\}$ \nÂngulo:  $\{ang\_graus:.2f\}^\circ$ )
```

```
# ##### 5. Produto Vetorial ( $v_1 \times v_2$ )
```

```
# Visualização do Produto Vetorial
```

```
vectors_to_plot = [  
    (v1, 'r', 'Vetor v1'),  
    (v2, 'g', 'Vetor v2'),  
    (produto_vetorial, 'b', 'Produto Vetorial ( $v_1 \times v_2$ )')  
]  
plot_vectors_3d(vectors_to_plot, title='Produto Vetorial:  $v_1 \times v_2$ )
```

```
# ##### 6. Vetor Unitário de v1
```

```
# Visualização do Vetor Unitário
```

```
vectors_to_plot = [
```

```
    (v1, 'r', 'Vetor v1'),
```

```
    (vetor_unitario_v1, 'g', 'Vetor Unitário de v1')
```

```
]
```

```
plot_vectors_3d(vectors_to_plot, title='Vetor Unitário de v1')
```